Figure 8.8 When Catalog Service doesn't respond successfully, Order Service will try at most three more times with a growing delay.

DEFINING RETRIES FOR WEBCLIENT

Project Reactor provides a `retryWhen()` operator to retry an operation when it fails. The position where you apply it to the reactive stream matters.

- Placing the `retryWhen()` operator after `timeout()` means that the timeout is applied to each retry attempt.
- Placing the `retryWhen()` operator before `timeout()` means that the timeout is applied to the overall operation (that is, the whole sequence of the initial request and retries has to happen within the given time limit).

In `BookClient`, we want the timeout to apply to each retry attempt, so we'll use the first option. The time limiter is applied first. If the timeout expires, the `retryWhen()` operator kicks in and tries the request again.

Update the `getBookByIsbn()` method in the `BookClient` class to configure a retry strategy. You can define the number of attempts and the minimum duration for the first backoff. The delay is computed for each retry as the current attempt number multiplied by the minimum backoff period. A jitter factor can be used to add randomness to the exponential of each backoff. By default, a jitter of at most 50% of the computed delay is used. When you have multiple instances of Order Service running, the jitter factor ensures that the replicas will not retry requests simultaneously.

Listing 8.25 Defining retries with exponential backoff for the HTTP call

```
public Mono<Book> getBookByIsbn(String isbn) {
  return webClient
```